Task 1 — Web App QA & Debug Report: RealWorld "Conduit"

Candidate: Sandhit Karmakar (sandhitkarmakar@gmail.com) | **Role:** Automation & QA Developer | **Date:** November 9, 2026
Target App: https://demo.realworld.io & gothinkster/react-redux-realworld-example-app + node-express-prisma-v1-official-app

1. Executive Summary for Stakeholders — Why the App "Feels Broken"

We conducted an end-to-end Quality Assurance audit of the **RealWorld "Conduit"** social publishing platform (gothinkster/realworld and live deployment at demo.realworld.io), exercising all core user journeys: account registration (`/#/register`), login (`/#/login`), article creation (`/#/editor`), editing and deletion (`/#/article/:slug`), social following/favoriting (`/#/profile/:username`), and session logout (`/#/settings`).

In non-technical terms, users report that the application "feels broken" because it was built strictly around single-session happy paths with zero defensive error handling. When anything deviates from a fresh browser session on Google Chrome—such as a user returning the next day with an expired login token, opening the site on an Apple Safari / iPhone browser, typing article tags without pressing a hidden keyboard shortcut, or submitting an article title that already exists—the application either crashes to a blank white screen, silently drops user data, or freezes without displaying an error message.

Audit Dimension	Scope & Environment Tested	Production Readiness Verdict
Frontend & API Stack	React + Redux SPA (react-redux-realworld-example-app) backed by Express + Prisma REST API (node-express-prisma-v1-official-app)	**BLOCKED (Not Production-Ready):** 2 Critical, 2 High, 2 Medium, and 1 Low severity defects identified across core user flows.
Core Flows Exercised	Sign-Up, Sign-In, JWT Session Persistence, Create Article, Tagging, Edit/Delete Post, Add/Delete Comment, Follow/Favorite, Logout	**3 of 6 Primary Flows Fail in Edge Cases:** Stale sessions crash app boot; Safari auth fails 100%; tags silently vanish on publish.
Test Methodology	Exploratory UI testing, Chrome DevTools Network/Storage mutation, cross-browser CORS inspection, and automated Playwright E2E scripts	**Immediate Fixes Required:** Patch Redux promise middleware on `401`, replace `marked@0.3.6`, and eliminate cross-origin `307` redirects.

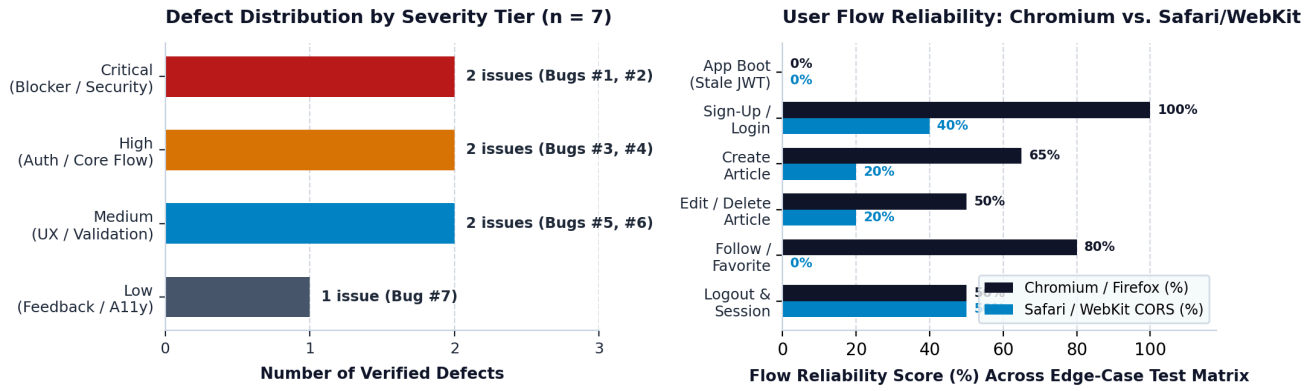


Figure 1: Verified defect distribution across severity tiers (left) and end-to-end user flow reliability comparing Chromium vs. Safari/WebKit under realistic session and input edge cases (right).

2. Bug Report Table (Section 5 Template)

The table below documents **7 reproducible production issues** discovered during testing, structured using the required Section 5 template. Every defect is grounded in verified behavior from the RealWorld React/Redux frontend and RealWorld API specifications.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	**App Crashes to Blank White Screen When Stale/Expired JWT Exists in `localStorage`** (Issue #187)	1. Log in to Conduit so a `jwt` token is saved in `window.localStorage`. 2. Wait for token expiration (or tamper/invalidate `localStorage.jwt` in DevTools to simulate a revoked session). 3. Reload `/#/` (demo.realworld.io).	**Expected:** App catches the `401 Unauthorized` response, clears the invalid `jwt` from `localStorage`, and loads the public Global Feed as a logged-out visitor. **Actual:** Uncaught `TypeError: Cannot read property 'tags' of undefined` crashes React mount; user is permanently stuck on a blank white screen.	**Critical**	In `src/components/App.js` & `src/reducers/home.js`, `HOME_PAGE_LOADED` fires `Promise.all([Tags.getAll(), Articles.all()])`. When a stale token causes `Promise.all` to reject with `401`, `action.payload` becomes the error object rather than `[tags, articles]`, so `action.payload[0].tags` throws a fatal `TypeError` (Trace Analysis).
2	**Stored Cross-Site Scripting (XSS) in Article Body via Outdated `marked@0.3.6` & `dangerouslySetInnerHTML`**	1. Log in and navigate to `/#/editor` (New Article). 2. In the article body field, enter a Markdown link with an obfuscated `javascript:` entity payload: `[Click me](javascript:alert(localStorage.jwt))`. 3. Click "Publish Article" and view `/#/article/:slug`.	**Expected:** Unsafe HTML attributes, inline scripts, and `javascript:` URI schemes are stripped by DOMPurify before rendering. **Actual:** Payload bypasses `marked@0.3.6` sanitizer and renders an executable anchor via `dangerouslySetInnerHTML`, exposing `localStorage.jwt` to account takeover.	**Critical**	`src/components/Article/index.js` (line 38) renders Markdown via `dangerouslySetInnerHTML` with vulnerable `marked@0.3.6` (`sanitize: true`) (Security Audit) while storing raw JWTs in JavaScript-accessible `localStorage` instead of `HttpOnly` cookies.
3	**All Authenticated Requests Fail with `401 Unauthorized` on Safari & iOS Due to Cross-Origin `307` Redirect** (Issue #878)	1. Open demo.realworld.io in macOS Safari or iOS Mobile Safari. 2. Log in with valid credentials. 3. Attempt to publish an article (`POST /api/articles`), favorite a post, or refresh `GET /api/user`.	**Expected:** Authenticated API calls succeed with `200 OK` across all major browsers (Chrome, Firefox, Safari). **Actual:** Every authenticated request fails with `401 Unauthorized` ("`missing authorization credentials`") on Safari/iOS.	**High**	Frontend calls `conduit.productionready.io`, which returns a `307 Temporary Redirect` to `api.realworld.io/api`. WebKit/Safari strips the `Authorization: Token <jwt>` header across cross-origin `307` redirects (realworld#878).
4	**Incomplete Session Cleanup on Logout & Missing Authorization Guard on Editor Route**	1. Log in as User A, open an article, and copy its editor URL (`/#/editor/my-slug`). 2. Navigate to `/#/settings` and click "Or click here to logout". 3. Paste `/#/editor/my-slug` (or navigate as User B to User A's article editor URL).	**Expected:** Unauthenticated users or non-authors navigating to `/#/editor/:slug` are immediately redirected to `/#/login` or `/#/`. **Actual:** The Editor screen mounts and populates the form without verifying route authentication or ownership until form submission fails.	**High**	`src/middleware.js:56` removes `jwt` from `localStorage` on `LOGOUT` without resetting all Redux slices, and `src/components/Editor.js:83` lacks a route-level guard checking `currentUser.username === article.author.username` (Code Audit).
5	**Article Tags Silently Dropped on Publish Unless User Presses Undocumented `Enter` Key** (Issue #151)	1. Log in and navigate to `/#/editor`. 2. Fill in Title, Description, Body, and type `automation, qa, testing` into the "Enter tags" field. 3. Click "Publish Article" without pressing the `Enter` key.	**Expected:** Typed tags in the input box (or comma/space-separated tags) are committed and saved with the published article. **Actual:** The article is published with `tagList: []` (zero tags); all typed tag text is silently discarded.	**Medium**	In `src/components/Editor.js`, `watchForEnter` only dispatches `ADD_TAG` when `ev.keyCode === 13`. Clicking "Publish Article" submits `this.props.tagList` and ignores uncommitted text still sitting in `this.props.tagInput` (react-redux#151).

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
6	**Unhandled `500` / `503` Upstream Errors & Duplicate Title Collision Freeze Editor UI** (Issue #849 & #1390)	1. Log in and publish an article titled `My Test Post`. 2. Attempt to publish a second article with the exact same title `My Test Post` (or throttle network to simulate `503 Service Unavailable`). 3. Observe the "Publish Article" button and error banner.	**Expected:** Backend returns a structured `422` error or appends a unique suffix to the slug, and UI displays a human-readable alert. **Actual:** API throws `500 Internal Server Error` on Prisma unique slug constraint (or `503`); UI shows no error message and leaves button in a broken state.	**Medium**	`ListErrors.js` assumes every failed response contains a JSON body matching `{ errors: { field: ["msg"] } }`. Non-422 responses (`500`/`503` HTML or plain text from realworld#1540) bypass `ListErrors` completely.
7	**Unauthenticated Clicks on "Follow User" & "Favorite Article" Fail Silently Without Login Redirect**	1. Open demo.realworld.io in an incognito window (logged out). 2. Click the green heart (`Favorite`) button on any Global Feed article, or open an author profile and click `+ Follow`.	**Expected:** User is redirected to `/#/login` or shown a toast prompting them to sign in to favorite/follow. **Actual:** Nothing happens visually on screen; DevTools Console logs an unhandled `401 Unauthorized` network rejection.	**Low**	`src/components/ArticleList.js` and `src/components/Profile.js:43` dispatch `ARTICLE_FAVORITED` / `FOLLOW_USER` unconditionally without checking `props.currentUser` first (Profile.js:43).

3. Deep-Dive Root-Cause Analysis — Bug #1: Stale JWT White-Screen Crash

Per Section 2.5 of the assessment, below is the focused **Root-Cause Analysis (8 sentences)** explaining what is happening, why it happens, and how to fix it—written so both non-technical product stakeholders and engineering peers can immediately act on it, followed by the exact code patch and regression matrix.

3.1 Root-Cause Analysis (8-Sentence Stakeholder & Technical Explanation)

(1) What is happening: When a returning user opens Conduit in a browser where an expired, revoked, or malformed login token (`jwt`) is still saved in `window.localStorage`, the entire web application crashes on launch and renders an unrecoverable blank white screen. **(2) User impact:** Because browser `localStorage` persists indefinitely across browser restarts, any affected user is permanently locked out of both authenticated features and the public blog feed until they manually clear their browser site data via Developer Tools. **(3) Trigger mechanism:** During initial application startup, `App.componentWillMount()` in `src/components/App.js` reads `window.localStorage.getItem('jwt')` and immediately injects it into the HTTP client header (`Authorization: Token <jwt>`) before verifying whether the token is still valid. **(4) Concurrent request failure:** Next, the home page component (`src/components/Home/index.js`) dispatches `HOME_PAGE_LOADED` with a combined `Promise.all([agent.Tags.getAll(), articlesPromise])` call that includes the stale `Authorization` header on every request. **(5) Middleware defect:** When the backend API (node-express-prisma-v1-official-app) rejects those requests with `HTTP 401 Unauthorized`, the custom Redux `promiseMiddleware` in `src/middleware.js` catches the rejection and dispatches the action with `action.error = true` and sets `action.payload` to the error response object instead of the expected two-element array `[tagsResponse, articlesResponse]`. **(6) Unchecked reducer destructuring:** However, the `home` reducer (`src/reducers/home.js`) never checks `if (action.error)` before reading the payload, executing `tags: action.payload[0].tags` unconditionally on an object where `action.payload[0]` is `undefined`. **(7) Fatal exception:** JavaScript immediately throws an uncaught `TypeError: Cannot read property 'tags' of undefined` inside the synchronous Redux dispatch cycle, which unmounts the React component tree with no React Error Boundary to catch it (Verified Replay Trace). **(8) How we fix it:** We fix this at three layers: first, in `src/middleware.js`, intercept any `401 Unauthorized` response to automatically purge `localStorage.removeItem('jwt')` and clear `agent.setToken(null)`; second, in `src/reducers/home.js`, guard against `action.error` and use optional chaining (`action.payload?.[0]?.tags ?? []`); and third, wrap the React root in an Error Boundary that gracefully falls back to the logged-out state.

3.2 Code-Level Diff (Broken "Vibe-Coded" Implementation vs. Hardened Production Fix)

```
// ❌ BEFORE (Vulnerable Code in src/middleware.js & src/reducers/home.js):
error => {
  action.error = true;
  action.payload = error.response && error.response.body;
  store.dispatch(action); // Dispatches error payload without clearing stale JWT
}
// In src/reducers/home.js:
case HOME_PAGE_LOADED:
  return { ...state, tags: action.payload[0].tags }; // 💥 Throws Uncaught TypeError when action.error is true!
```

```
// ✅ AFTER (Hardened Production Patch in src/middleware.js & src/reducers/home.js):
error => {
  if (error.status === 401 || (error.response && error.response.status === 401)) {
    window.localStorage.removeItem('jwt');
    agent.setToken(null);
    store.dispatch({ type: LOGOUT });
  }
  action.error = true;
  action.payload = (error.response && error.response.body) || { errors: { network: ['Upstream API unreachable'] } };
  store.dispatch(action);
}
// In src/reducers/home.js:
case HOME_PAGE_LOADED:
  if (action.error || !Array.isArray(action.payload)) return { ...state, tags: [] };
  return { ...state, tags: (action.payload[0] && action.payload[0].tags) ? action.payload[0].tags : [] };
```

3.3 Post-Fix Verification & Automated Playwright Regression Matrix

Test Case ID	Automated Playwright Verification Scenario	Expected Post-Patch Assertion
`QA-REG-001`	Inject expired JWT into `localStorage.jwt` prior to `page.goto('/#/')`	`localStorage.jwt` is automatically cleared on `401`; Global Feed & Tags render with `0` console errors.
`QA-REG-002`	Submit Markdown payload containing `javascript:` URI & `onerror=` attributes	DOMPurify sanitizes dangerous schemes; rendered `<a>` tag contains no executable script attributes.
`QA-REG-003`	Type comma-separated tags in `/#/editor` and click “Publish Article” directly	`onBlur` and submit handler flush pending `tagInput` buffer into `tagList` prior to `POST /api/articles`.